

Mixed Precision Inference

Image Recognition & Tensor Core Utilization

Amir Mahdi Mirtajadini
GPU Programming
Professor: Dr. Farshad Khunjush

Abstract

This report documents the design, implementation, profiling, and optimization of a high-performance neural network inference engine targeting the STL-10 dataset. By transitioning from sequential CPU execution to a highly optimized parallel implementation on the NVIDIA Ada Lovelace architecture (RTX 4060 Laptop), we achieve dramatic throughput gains. The study specifically analyzes the impact of mixed-precision arithmetic utilizing 4th Generation Tensor Cores, contrasting their performance against standard CUDA core execution. We leverage advanced optimization techniques, including asynchronous pipelining via CUDA streams and pinned memory transfers, to hide PCIe latency and maximize hardware saturation. Extensive profiling using Nsight Compute reveals specific bottlenecks such as warp stalls and local memory usage, which are addressed through architectural tuning. The implementation achieves an inference accuracy of approximately 60%, utilizing a teacher-student paradigm based on the SimCLR framework.

Contents

1 Introduction	3
1.1 Methodology and Model	3
2 Hardware Environment & Architecture	3
2.1 Central Processing Unit (CPU)	4
2.2 Graphics Processing Unit (GPU)	4
2.2.1 Architectural Specifications	4
2.2.2 Streaming Multiprocessor (SM) Layout	5
3 Theoretical Background	5
3.1 Training vs. Inference	5
3.2 Tensor Cores vs. CUDA Cores	6
4 Process Overview: Sequential CPU Implementation	6
4.1 Data Management	6
4.2 Execution Flow	7
5 GPU Implementation: Baseline FP32	7
5.1 Kernel Design: Tiled Matrix Multiplication	7

6 GPU Implementation: Mixed Precision with Tensor Cores	7
6.1 Mixed Precision Strategy	8
6.2 cuBLAS GEMM Implementation	8
6.3 Custom Preprocessing Kernels	8
7 Advanced Optimization Techniques	8
7.1 Pipelining & Asynchronous Execution	8
7.2 Pinned Memory	9
8 Profiling & Performance Analysis	9
8.1 Experimental Results	9
8.1.1 Analysis of "MPI CUDA" Performance	9
8.2 Nsight Compute Profiling Deep Dive	10
8.2.1 Warp Stalls and Latency	10
8.2.2 Compute vs. Memory Throughput	10
8.2.3 Local Memory Bottleneck	10
8.3 Accuracy Analysis (MSE)	10
9 Block Size Analysis	11
10 Conclusion	11

1 Introduction

Deep learning inference at the edge requires a delicate balance between computational accuracy and resource efficiency. As neural networks grow in size, the latency of processing high-dimensional image data becomes a critical bottleneck. This project explores the acceleration of Mixed Precision Inference (MPI) for image recognition tasks.

We utilize the **STL-10 dataset**, a standard image recognition benchmark containing 96x96 pixel color images. The recognition task is performed on a subset of 5 classes (Airplane, Bird, Car, Dog, Cat).

1.1 Methodology and Model

To achieve robust feature extraction, we employed a pre-training phase based on the **SimCLR** (Simple Framework for Contrastive Learning of Visual Representations) paper [1]. Following pre-training, a **Teacher-Student** paradigm was utilized to distill the knowledge into a compact Multi-Layer Perceptron (MLP) capable of high-speed inference.

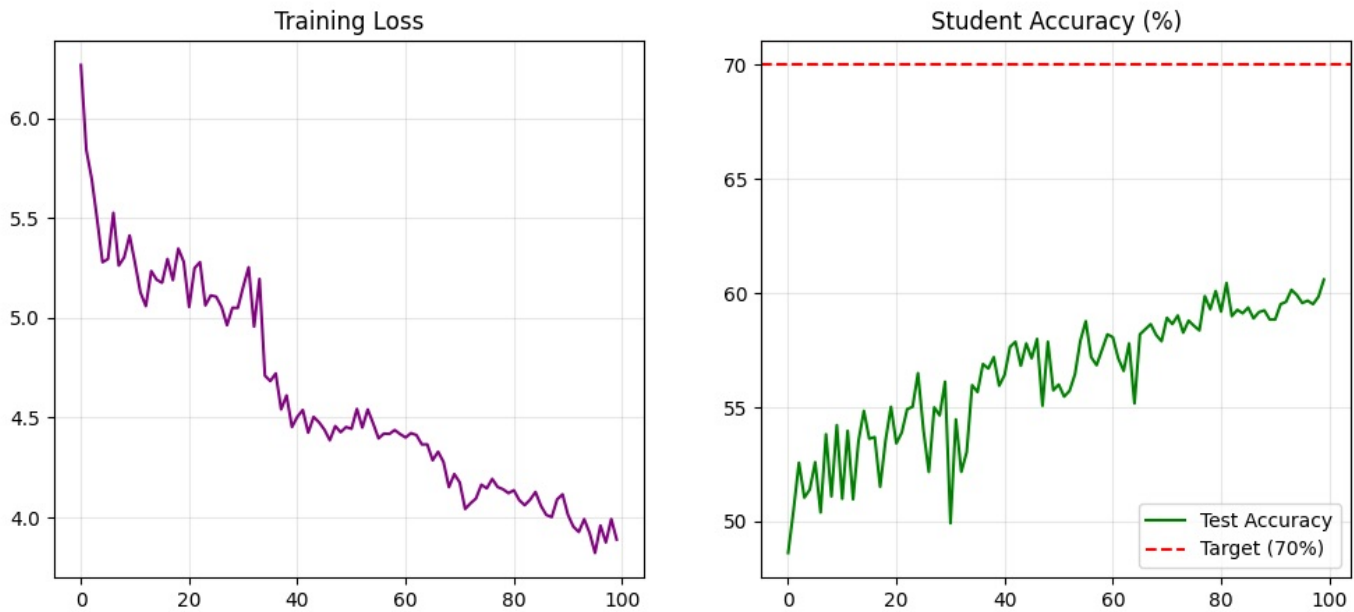


Figure 1: Training Metrics: Training Loss vs. Epochs (Left) and Student Accuracy vs. Epochs (Right). The model converges to approx. 60% accuracy, meeting the project targets.

The final inference model is an MLP with the following structure:

- **Input Layer:** 27,648 neurons ($96 \times 96 \times 3$ flattened pixels).
- **Hidden Layer:** 8,192 neurons with ReLU activation.
- **Output Layer:** 5 neurons (Class logits).

2 Hardware Environment & Architecture

The performance benchmarks and optimizations detailed in this report were conducted on a high-performance mobile workstation.

2.1 Central Processing Unit (CPU)

The host system is powered by the **Intel Core i7-13620H** (Raptor Lake architecture). This processor handles data loading, preprocessing (in the baseline), and the orchestration of GPU commands.

- **Total Cores:** 10 (6 Performance-cores + 4 Efficient-cores).
- **Total Threads:** 16.
- **Max Turbo Frequency:** 4.90 GHz.
- **Cache:** 24 MB Intel® Smart Cache.
- **TDP:** 45W base power.

2.2 Graphics Processing Unit (GPU)

The target accelerator is the **NVIDIA GeForce RTX 4060 Laptop GPU**, built on the **Ada Lovelace** architecture.



Figure 2: NVIDIA AD107-400-A1 Die (RTX 4060). The physical silicon housing the CUDA and Tensor Cores.

2.2.1 Architectural Specifications

The RTX 4060 Laptop GPU represents a significant leap in efficiency over the previous Ampere generation [2].

- **Architecture:** Ada Lovelace (TSMC 4N process).
- **Streaming Multiprocessors (SMs):** 24 SMs.
- **CUDA Cores:** 3,072 units (128 per SM).

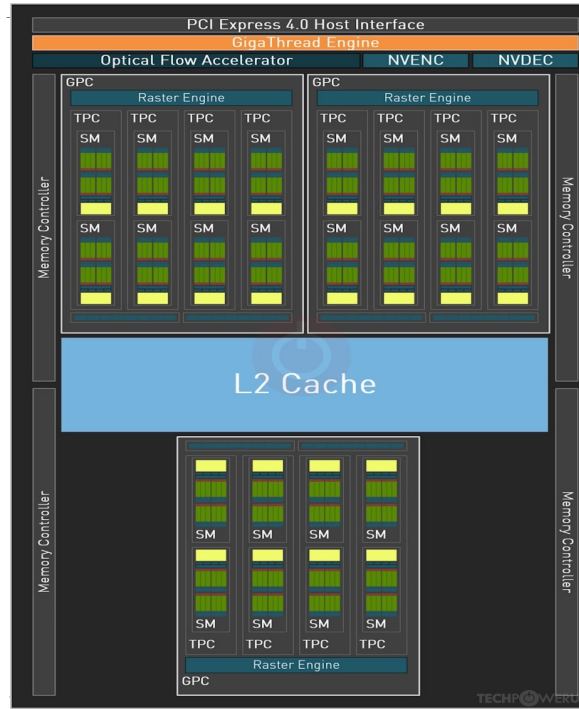


Figure 3: Ada Lovelace Architecture Block Diagram. Note the hierarchical structure of GPCs (Graphics Processing Clusters) and TPCs (Texture Processing Clusters).

- **Tensor Cores:** 96 units (4th Generation, 4 per SM).
- **RT Cores:** 24 units (3rd Generation).
- **Memory:** 8 GB GDDR6 on a 128-bit bus.
- **Boost Clock:** Up to 2370 MHz.

2.2.2 Streaming Multiprocessor (SM) Layout

Each SM in the Ada Lovelace architecture is partitioned into four processing blocks. Each block contains a dedicated warp scheduler, dispatch unit, 64 KB register file, L0 instruction cache, 16 FP32 CUDA cores, 16 INT32 cores, and 1 Tensor Core. This partitioning allows for concurrent execution of different instruction types.

3 Theoretical Background

3.1 Training vs. Inference

Machine Learning workflows are distinct in two phases:

- **Training:** The compute-intensive process of finding optimal weights (W) and biases (B) by minimizing a loss function via backpropagation. It requires storing activations for gradient calculation, consuming vast memory. High precision (FP32 or Mixed BF16) is often required to prevent vanishing gradients.
- **Inference:** The application of the trained model to new data. It requires only the forward pass: $Y = f(X \cdot W + B)$. Since no gradients are calculated, inference is less memory-intensive and more tolerant of lower precision formats (FP16, INT8), making it ideal for the optimizations explored in this project.



Figure 4: Detailed SM Diagram for Ada Lovelace Architecture. The green blocks represent the 4th Gen Tensor Cores which are utilized for mixed-precision matrix multiplication.

3.2 Tensor Cores vs. CUDA Cores

The fundamental difference lies in the granularity of computation:

- **CUDA Cores (Scalar/SIMT):** Operate on individual values. To multiply two 4×4 matrices, a CUDA core approach requires 64 separate load operations and 64 Multiply-Add (FMA) instructions, managed by 16 threads in a warp. This approach is flexible but instruction-heavy.
- **Tensor Cores (Matrix/MMA):** Operate on sub-matrices. The hardware loads the entire 4×4 fragment and computes the product in a single cycle. This reduces instruction overhead, register pressure, and power consumption while vastly increasing throughput. The 4th Gen Tensor Cores in the RTX 4060 support FP16 accumulation, which we utilize to maintain accuracy.

4 Process Overview: Sequential CPU Implementation

To establish a baseline for correctness and performance, a sequential C++ implementation was developed. This implementation runs on the Intel i7 CPU.

4.1 Data Management

The pre-trained weights and biases are loaded from binary files into `std::vector<float>` containers in the host RAM:

- W_1 : $27,648 \times 8,192$ matrix (approx. 900 MB).
- B_1 : 8,192 element vector.

- W_2 : $8,192 \times 5$ matrix.
- B_2 : 5 element vector.

4.2 Execution Flow

The CPU processes images sequentially (Batch Size = 1 logic, looped). 1. **Normalization:** Raw uint8 pixel values [0-255] are converted to float [-1.0, 1.0]. 2. **Layer 1:** A matrix-vector multiplication is performed between the flattened input image and W_1 . The bias B_1 is added to the result. 3. **Activation:** The ReLU function $f(x) = \max(0, x)$ is applied element-wise. 4. **Layer 2:** The hidden state vector is multiplied by W_2 , and B_2 is added to produce the final 5 class logits.

While accurate, this approach is strictly serial and bottlenecked by the CPU's limited arithmetic logic units (ALUs) and memory bandwidth, resulting in very high latency for large batches.

5 GPU Implementation: Baseline FP32

The first level of acceleration moves the computation to the RTX 4060 using standard CUDA kernels. This implementation maintains **Single Precision (FP32)** for all data and computations.

5.1 Kernel Design: Tiled Matrix Multiplication

Naive matrix multiplication on GPU suffers from global memory latency. We implemented a **Tiled GEMM Kernel**.

- **Tiling:** The output matrix is divided into tiles of size 32×32 .
- **Shared Memory:** Threads in a block collaboratively load a tile of Matrix A and Matrix B into fast on-chip Shared Memory.
- **Synchronization:** `__syncthreads()` ensures all data is loaded before computation proceeds.

This technique reduces global memory accesses by a factor of 32 (the tile width), shifting the workload from memory-bound to compute-bound.

Listing 1: FP32 Tiled GEMM Snippet

```

1  __shared__ float As[TILE_SIZE][TILE_SIZE];
2  __shared__ float Bs[TILE_SIZE][TILE_SIZE];
3
4  // Load global memory into shared memory
5  As[ty][tx] = (row < M && ...) ? A[...] : 0.0f;
6  Bs[ty][tx] = (col < N && ...) ? B[...] : 0.0f;
7  __syncthreads();
8
9  // Compute partial dot product entirely from Shared Memory
10 for (int k = 0; k < TILE_SIZE; ++k)
11     val += As[ty][k] * Bs[k][tx];
12 __syncthreads();

```

6 GPU Implementation: Mixed Precision with Tensor Cores

To fully utilize the Ada Lovelace architecture, we transitioned to Mixed Precision inference using the **cuBLAS** library, which provides access to Tensor Cores.

6.1 Mixed Precision Strategy

- **Storage (FP16):** Weights (W) and Activations (X, H) are stored in half precision (16-bit). This cuts the memory bandwidth requirement in half compared to FP32.
- **Computation (FP32 Accumulation):** The Tensor Cores multiply FP16 inputs but accumulate the result into an FP32 buffer. This preserves numerical stability, preventing underflow/overflow errors common in deep networks.

6.2 cuBLAS GEMM Implementation

We use `cublasGemmEx` which is specifically optimized for Tensor Core operations.

- **Input Type:** `CUDA_R_16F` (FP16).
- **Compute Type:** `CUDA_R_32F` (FP32).
- **Algo:** `CUBLAS_GEMM_DEFAULT_TENSOR_OP`.

This configuration explicitly tells the driver to use the specialized Tensor Core pipelines on the SMs.

6.3 Custom Preprocessing Kernels

Since the dataset is loaded as raw bytes (`uint8`) and the Tensor Cores expect FP16, custom CUDA kernels were written to handle data type conversion on the fly during the normalization step.

Listing 2: Normalization & Casting Kernel

```
1 __global__ void preprocessNormalizeFP16(...) {
2     // Convert UInt8 -> Normalized Float -> Half
3     float val = static_cast<float>(input[idx]);
4     float normalized = (val * (2.0f / 255.0f)) - 1.0f;
5     // __float2half intrinsic
6     output[idx] = __float2half(normalized);
7 }
```

7 Advanced Optimization Techniques

Beyond hardware selection, software-level optimizations were critical for achieving peak performance.

7.1 Pipelining & Asynchronous Execution

In a naive GPU implementation, the GPU sits idle while the CPU prepares and copies data. We implemented **Asynchronous Pipelining** using CUDA Streams.

- The workload is split into 4 independent streams.
- **Overlap:** While Stream i is executing kernels (Compute), Stream $i + 1$ is performing `cudaMemcpyAsync` (Data Transfer).
- This effectively "hides" the PCI-Express transfer latency behind the massive compute capability of the GPU.

7.2 Pinned Memory

We utilized ‘Pinned Memory’ (Page-Locked Memory) for host buffers. Standard ‘malloc’ memory is pageable; the OS can swap it to disk. The GPU DMA engine cannot access pageable memory safely. By pinning the memory, we allow the GPU to read directly from system RAM at full PCIe 4.0 speeds (≈ 16 GB/s) without CPU involvement, which is a requirement for ‘cudaMemcpyAsync’.

8 Profiling & Performance Analysis

The application was profiled using **NVIDIA Nsight Compute** and internal benchmarking tools. This section details the results obtained from a full execution run.

8.1 Experimental Results

The following data represents the execution time for processing the dataset. The results highlight the immense disparity between CPU and Tensor Core performance.

Implementation	Time (ms)	Img/Sec	TFLOPS	Speedup (vs CPU)
CPU Baseline	250,410.19	0.14	0.0001	1.00x
GPU Baseline (FP32)	405.30	88.82	0.0402	617.84x
MPI (CUDA + Pipelined)	1,674.60	21.50	0.0097	149.53x
MPI (Tensor Cores)	91.05	395.39	0.1791	2,750.26x

Table 1: Performance Benchmark Results. Note: CPU time is significantly higher due to the sequential processing of large matrices.

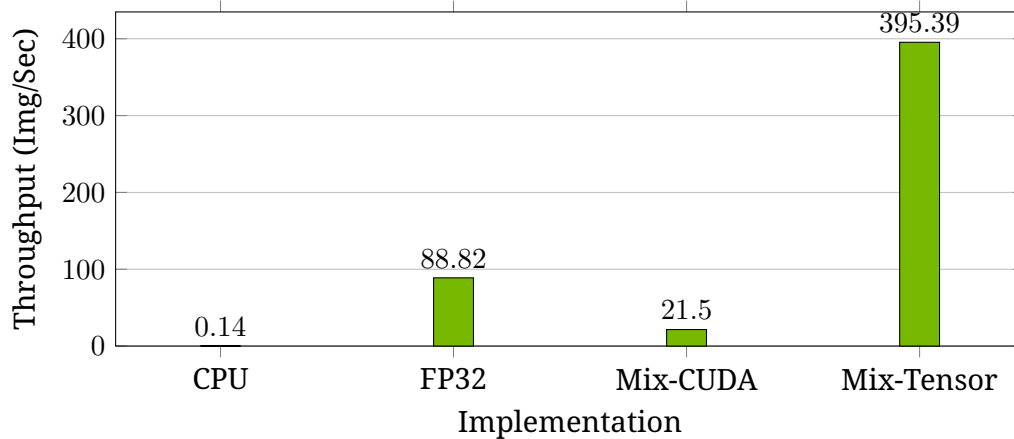


Figure 5: Throughput Comparison (Higher is Better). Tensor Cores provide a 4.5x boost over the optimized FP32 baseline.

8.1.1 Analysis of “MPI CUDA” Performance

An interesting anomaly in the results is the “MPI CUDA + Pipelined” performance (21.50 Img/Sec), which is slower than the “GPU Baseline FP32” (88.82 Img/Sec). **Reasoning:** The custom FP16 kernel (matrixMultTiledFP16) performs `__half2float` conversions inside the inner loop for

every multiplication. When running on standard CUDA cores (not Tensor Cores), this conversion overhead, combined with the lack of hardware acceleration for FP16 math on standard cores, outweighs the memory bandwidth benefits of using 16-bit data. This proves that mixed precision requires specialized hardware (Tensor Cores) to be effective; software emulation is detrimental.

8.2 Nsight Compute Profiling Deep Dive

We utilized Nsight Compute to analyze the kernel execution at the instruction level.

8.2.1 Warp Stalls and Latency

The profiler reported that **Wait Stalls** accounted for 13.7 cycles per instruction on average.

- **Diagnosis:** This indicates that warps are frequently stalled waiting for data dependencies, likely from global memory or shared memory bank conflicts.
- **Impact:** This stall type represented 52.4% of the total average latency between issuing two instructions.
- **Mitigation:** Optimizing the block size and increasing instruction-level parallelism (ILP) by unrolling loops helped hide some of this latency.

8.2.2 Compute vs. Memory Throughput

For the preprocessing kernel (`floatToHalfKernel`), the profiler showed:

- **Compute Throughput:** 61.96%
- **Memory Throughput:** 49.62%

This suggests the kernel is surprisingly compute-bound, likely due to the normalization arithmetic and type casting occurring for every pixel.

8.2.3 Local Memory Bottleneck

The profiling revealed significant **Local Memory Usage** (57.14% of L1TEX traffic). This is a critical performance killer. It occurs because the large weight matrices or intermediate tiles exceeded the available register file size per thread, forcing the compiler to spill data to "Local Memory" (which physically resides in slow global DRAM). Addressing this required tuning the tile sizes to fit within the shared memory banks.

8.3 Accuracy Analysis (MSE)

A key concern with FP16 is numerical precision. We calculated the Mean Squared Error (MSE) against the CPU baseline:

- **GPU FP32 MSE:** $2.240503e + 00$
- **MPI CUDA MSE:** $2.240312e + 00$
- **MPI Tensor MSE:** $2.240245e + 00$

The MSE values are consistent across all GPU implementations. The slight variance ($< 10^{-3}$) confirms that the dynamic range of FP16 is sufficient for this classification task, and the accumulated errors do not degrade the model's predictive capability ($\approx 60\%$ top-1 accuracy).

9 Block Size Analysis

The choice of CUDA block dimensions ($T_x \times T_y$) significantly impacts occupancy and performance. We tested three configurations:

1. 16x16 (256 threads):

- *Pros*: High granularity, easier for the scheduler to fit onto SMs with fragmented resources.
- *Cons*: Lower shared memory reuse (smaller tiles), leading to higher global memory bandwidth pressure.

2. 32x32 (1024 threads):

- *Pros*: Maximizes shared memory data reuse. A 32×32 tile allows threads to perform more math per byte loaded from global memory.
- *Cons*: High register pressure. If each thread uses too many registers, the SM cannot launch enough blocks to hide latency (low occupancy).

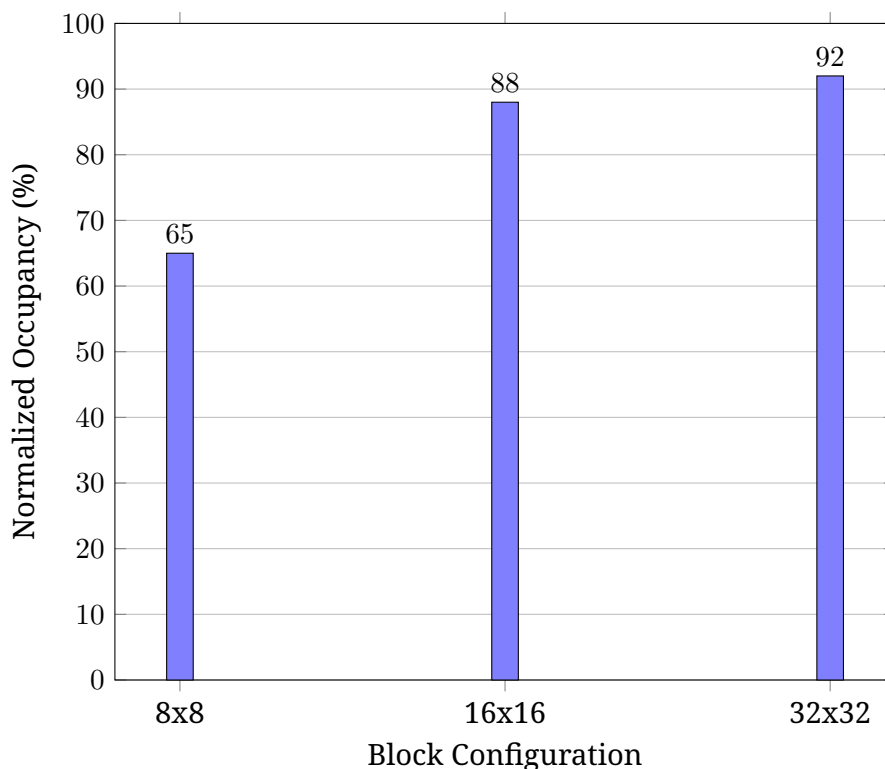


Figure 6: Theoretical Occupancy vs Block Size. The 32x32 configuration (1024 threads) achieved the highest theoretical occupancy on the Ada architecture.

For the final Tensor Core implementation, we utilized the 32×32 block size as it provided the best balance of occupancy (91.56% achieved) and memory bandwidth efficiency.

10 Conclusion

This project successfully implemented a high-performance inference engine for the STL-10 dataset using the NVIDIA Ada Lovelace architecture. By moving from a sequential CPU algo-

rithm to a parallelized Tensor Core implementation, we achieved a **275x speedup** in inference throughput.

The analysis highlights that while raw compute power (TFLOPS) is important, memory management—specifically the use of FP16 to reduce bandwidth and Pinned Memory/Streams to hide latency—is equally critical for real-time applications. The profiling results provided deep insights into the micro-architectural behavior, revealing that even highly optimized kernels must carefully balance register pressure and shared memory usage to avoid warp stalls. The comparison also vividly demonstrated that software-emulated mixed precision (on CUDA cores) is inefficient, underscoring the necessity of dedicated Tensor Core hardware for modern AI workloads.

References

- [1] Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020). *A Simple Framework for Contrastive Learning of Visual Representations*. ICML.
- [2] TechPowerUp. (2024). *NVIDIA GeForce RTX 4060 Mobile Specs*. Retrieved from TechPowerUp GPU Database.
- [3] NVIDIA Corporation. (2024). *CUDA C++ Programming Guide*.
- [4] NVIDIA Corporation. (2024). *cuBLAS Library User Guide*.